

TP I : Quantification, convolution, TFD

1- Rappels théoriques:

A- Quantification

La quantification, étape qui suit l'acquisition, consiste à quantifier les échantillons disponibles c'est à dire à choisir un code numérique j choisi parmi J codes pré-spécifiés pour représenter toutes les valeurs des échantillons. Le nombre de codes est en général une puissance de deux ($J = 2^B$) de sorte qu'on puisse représenter un code j sur un mot binaire de longueur B . Pour une quantification linéaire, la précision d'une amplitude variante entre $-V_{\max}$ et $+V_{\max}$ est :

$$q = \frac{V_{\max} - V_{\min}}{2^B} = \frac{2V_{\max}}{2^B}$$

La restitution du signal consiste à retrouver son amplitude à partir des j niveaux, résultat que l'on veut aussi proche que possible du signal d'entrée. Elle peut se faire de plusieurs manières. On parlera d'une restitution avec ou sans demi-échelon selon qu'on restitue ou non la valeur moitié de la plage de quantification q .

Si le signal d'entrée vérifie $|x_A(t)| < 1 \text{ V}$, une quantification uniforme sur $J = 2^B$ niveaux

donne $j = E\left(\frac{x_N(n)}{q}\right)$ avec $q = \frac{2}{J}$; q est le pas de quantification et E est la partie entière.

Le signal restitué est donné par: $x_{Nq}(n) = q\left(j + \frac{1}{2}\right)$ où $j=1,2,..., J$

B- Convolution de signaux discrets

Le produit de convolution des signaux numériques est donné par :

$$y(n) = x(n) * g(n) = \sum_{m=-\infty}^{+\infty} x(m)g(n-m)$$

Le signal $g(m)$ est inversé puis décalé de n

C- Transformée de Fourier discrète

• **Définition théorique :** La **TFD** est définie traditionnellement par une somme. Au signal discret $x_p[n]$ correspond une transformée discrète $X_p[k]$, calculée par :

$$X_p[k] = \sum_{n=0}^{N-1} x_p[n] e^{-j \frac{2\pi}{N} kn}$$

La **TFD inverse** est donnée par : $x_p[n] = \sum_{k=0}^{N-1} X_p[k] e^{+j \frac{2\pi}{N} kn}$

Où N est le nombre d'échantillons disponibles et $0 \leq k \leq N$

- Pour calculer la transformée de Fourier complexe d'un vecteur de longueur N on utilise la commande Matlab **fft** (pour la TFD inverse on utilise la commande **ifft**) sur les N points, soit de 0 à N-1 correspondant à l'intervalle $[0, F_e]$.
- Le spectre obtenu sera un vecteur complexe que l'on pourra représenter en coordonnées cartésiennes (partie réelle et imaginaire fonction **real** et **imag**), ou en coordonnées polaires (module et phase, fonction matlab **abs** et **angle** ou **unwrap**).
- Pour une représentation entre $-F_e/2$ et $F_e/2$ de la TFD, il suffira d'utiliser la commande **fftshift** ce qui permet de centrer la TFD.

Algorithme rapide de calcul : L'algorithme rapide de calcul de la TFD s'appelle la Transformation de Fourier Rapide (TFR) (Fast Fourier Transform (FFT) en anglais). impose comme contrainte que N (le nombre de points de la transformée) soit une puissance de 2. La TFR réduit le nombre d'opérations nécessaires au calcul de la TFD. La complexité de la TFD est en effet ramenée de N^2 (avec la définition théorique) à $N \cdot \log_2(N)$ en utilisant la TFR (voir cours).

2- Simulation sur Matlab

2.1 Etude de la quantification

1. On considère comme signal un extrait de la Mélodie Hongroise de Franz Schubert, charger ce signal en utilisant la commande **wavread** puis interpréter les informations affichées et vérifier que le signal a une amplitude égale au plus à 1 (commandes **min** et **max**).
2. Sélectionner 1024 échantillons du vecteur généré et en faire une représentation graphique.
3. Ecrire une fonction **Matlab** réalisant l'opération de quantification linéaire (la première ligne du programme est le nom de la fonction : **xq = quantlin (signal,B)**, pour cela utiliser la fonction **floor**. Visualiser le résultat pour différentes valeurs de B et interpréter.
4. Ecrire une fonction **Matlab** (fichier **testlin.m**) calculant le bruit de quantification $(x(n) - x_q(n))$ qu'on représentera graphiquement, et la puissance moyenne du signal d'erreur

de quantification $e = \frac{1}{N} \sum_{n=0}^{N-1} (x(n) - x_q(n))^2$ pour différentes valeurs de $J = 2^B$ avec $B = \{4, 8, 12\}$ et $N=10000$.

5. Refaire les opérations précédentes avec la fonction **quantlog** fournie, comparer et interpréter les résultats.

2.2 Produit de convolution de deux signaux discrets

Considérant un signal $x(n)$ de longueur N et un signal $g(n)$ de longueur M .

- 1- Proposer une routine Matlab qui met en œuvre la convolution discrète (pour faire le reversement temporel, utiliser la commande **fliplr**).
- 2- Utiliser cette routine pour le calcul du produit de convolution des deux signaux $x(n)$ et $g(n)$

donnés par :

$$x(n) = \begin{cases} 1 & \text{pour } 0 \leq n \leq N-1 \\ 0 & \text{ailleurs} \end{cases} \quad g(n) = \begin{cases} 0.7^n & \text{pour } 0 \leq n \leq M-1 \\ 0 & \text{ailleurs} \end{cases}$$

On prendra $N=10$ et $M=2N$

- 3- Vérifier que la longueur du signal $y(n) = x(n) * g(n)$ est de « $M+N-1$ » ;
- 4- Proposer une routine Matlab pour le calcul de la convolution linéaire en utilisant la **fft**.
- 5- Répéter le même travail pour les séquences suivantes : $N=3$ et $M=6$

$$f(n) = \begin{cases} \exp(0.1n) & \text{pour } 0 \leq n \leq N-1 \\ 0 & \text{ailleurs} \end{cases} \quad h(n) = \begin{cases} \exp(-0.1n) & \text{pour } 0 \leq n \leq M-1 \\ 0 & \text{ailleurs} \end{cases}$$

2.3 Application de la TFD

a. Ecrire une fonction **Matlab** réalisant La TFD d'un signal quelconque, en utilisant la définition de la TFD puis l'appliquer aux signaux suivants:

- *L'impulsion de Dirac* définie par: $d=[1, \text{zeros}(1,N-1)]$;
- *L'impulsion de Dirac* décalée définie par: $d=[\text{zeros}(1,N-1), 1, \text{zeros}(1, M-1)]$;
- *A $N=512$ échantillons* pris dans la somme de trois sinusoïdes de fréquences 10, 15 et 30Hz. Puis à $2N$ échantillons de la même somme. Conclure.
- Appliquer la fonction TFD, à un ensemble de 1024 échantillons du signal obtenu par la lecture d'un fichier .WAV (exemple : 'Schubert')

b. Utilisation de la FFT :

Utiliser la fonction **FFT** de **MATLAB** au même signal y et comparer le résultat obtenu avec celui obtenu par la fonction **TFD**.

Changer le nombre d'échantillons et refaire le même travail, comparer les résultats obtenus pour 512, et 2048 échantillons. Avec les commandes **TIC** et **TOC** de **MATLAB** puis donner les vitesses de calcul de la TFD et de la FFT. Conclure.

TP II: FILTRAGE DES SIGNAUX

Les filtres ont pour rôle de modifier les données vues par leurs entrées pour répondre à des spécifications fréquentielles ou autres. La transmittance (isochrone) d'un filtre numérique s'exprime selon l'équation générale : $H(\exp(j\phi)) = A(\phi) \exp(j\beta(\phi))$, $\phi = \omega T_e$

où T_e est la période d'échantillonnage, $A(\phi)$ est l'amplitude du filtre et $b(\phi) = -\beta(\phi)$ le déphasage introduit par ce filtre.

Le plus souvent, les filtres sont spécifiés dans le domaine fréquentiel $H(f)$. Les réponses fréquentielles idéalisées qui sont d'usage sont données uniquement sur la période principale $[-1/2, 1/2]$ de $H(f)$.

Les spécifications pratiques des filtres réalisables ne peuvent être données que par des approximations: intervalles de tolérance ou gabarit que doit satisfaire la réponse fréquentielle d'un filtre.

La spécification d'un filtre peut aller de la simple élimination d'une ou de plusieurs composantes spectrales, à la spécification avec contraintes sur les amplitudes des ondulations, sur les bandes passantes ou coupantes, sur la transition entre ces bandes ...etc. Cette spécification peut être encore plus précise si on y trouve d'autres contraintes supplémentaires telles que la nature du filtre ou son ordre.

On distingue les filtres dits à réponse impulsionnelle finie (RIF) et les filtres à réponse impulsionnelle infinie (RII). Les filtres à RII comportent moins de paramètres que les filtres à RIF mais présentent en général une phase non linéaire.

1- Filtres RII classiques:

Les quatre catégories des filtres idéaux sont les filtres passe-bas, passe-haut, passe-bande et coupe- bande. Il est toutefois suffisant de disposer des réponses fréquentielles du filtre passe-bas car, à partir de ces prototypes normalisés (pulsation de coupure égale à 1), on aboutit par simple changement de variable, aux autres catégories de filtres.

Dans la suite, on se restreindra aux filtres passe-bas. Les caractéristiques élémentaires des filtres proposés par MATLAB sont:

- *Filtre de Butterworth*: Meilleure approximation par les séries de Taylor de la réponse idéale d'un filtre passe bas entre 0 et l'infini. Il est connu par une amplitude constante dans la bande passante. Amplitude d'ondulation presque nulle dans la bande passante et

décroissante par ailleurs. Seuls les paramètres ordre du filtre et fréquence de coupure sont à préciser.

- *Filtre de Chebyshev de type I*: Minimise la différence en valeur absolue des réponses idéale et réelle sur la totalité de la bande passante. Il introduit des ondulations de même amplitudes R_p dans la bande passante.
- *Filtre de Chebyshev de type II*: Minimise la différence en valeur absolue des réponses idéale et réelle sur la totalité de la bande coupée. Il introduit des ondulations de même amplitude R_s dans la bande coupée. On parle de filtre de Chebyshev inverse.

2- Conception des filtres à RII:

Les filtres à RII digitaux proposés par MATLAB "*butter*", "*cheby1*", "*cheby2*", "*ellip*" et "*bessel*" sont relatifs aux filtres théoriques de Butterworth, de Chebyshev du type I et II, elliptiques et de Bessel respectivement. Ils ont été conçus sur la base de la version analogique des filtres passe-bas, normalisés à une fréquence de coupure de 1Hz, qui ont été transformés en des filtres d'autres types (passe-haut, passe-bande,...) puis discretisés.

Les arguments possibles à ces fonctions sont l'ordre du filtre, l'amplitude des ondulations dans une bande donnée, la fréquence de coupure et une option ('*high*' ou '*stop*') qui spécifie la nature du traitement (passante ou atténuante) du filtre.

Le tableau ci-dessous donne les expressions des commandes MATLAB pour les différents filtres :

Forme générique	Exemple	Options
$[b,a]=\text{butter}(n,W_n,\text{options})$	$[b,a]=\text{butter}(5,0.4);$	Pas d'option
$[b,a]=\text{ellip}(n,R_p,R_s,W_n,\text{option}).$	$[b,a]=\text{ellip}(3,1,60,[0.4 \ 0.7],\text{'stop'});$	Coupe-bande
$[b,a]=\text{cheby1}(n,R_p,W_n,\text{option}).$	$[b,a]=\text{cheby1}(4,1,[0.4 \ 0.7]);$	Pas d'option
$[b,a]=\text{cheby2}(n,R_s,W_n,\text{option}).$	$[b,a]=\text{cheby2}(6,60,0.8,\text{'high'});$	Passe-haut

où n est l'ordre du filtre, R_p et R_s représentent le niveau de l'amplitude permise des ondulations en dB dans la bande passante et dans la bande coupée respectivement.

W_n est la fréquence de coupure du filtre considéré par défaut comme un passe-bas (l'argument type est alors inutile). Si $W_n=[W_1 \ W_2]$ est un vecteur à 2 éléments, le filtre est considéré comme un passe-bande entre W_1 et W_2 .

L'argument type (lorsqu'il est utilisé) peut prendre soit la mention '*high*' pour un filtre passe-haut ou alors '*stop*' pour un filtre coupe-bande entre deux pulsations $[W_1 \ W_2]$.

L'élaboration de la version analogique de l'un de ces filtres est possible en ajoutant l'argument 's' pour signaler le plan analogique s, selon les exemples

[b,a]=butter(n,Wn,option,'s') ou **[b,a]=ellip(n,Rp,Rs,Wn,'type','s')**

Pour tous les filtres présentés, les commandes MATLAB donnent deux vecteurs de coefficients [b, a] constituant le numérateur et le dénominateur du filtre :

- **Dans le domaine discret**, les vecteurs coefficients du filtre sont donnés dans l'ordre décroissant des puissances de Z selon l'écriture.

$$H(z) = \frac{B(z)}{A(z)} = \frac{b(1) + b(2)z^{-1} + \dots + b(n+1)z^{-n}}{1 + a(2)z^{-1} + \dots + a(n+1)z^{-n}}$$

- **Dans le domaine analogique**, les vecteurs coefficients sont donnés dans l'ordre décroissant des puissances de s.

$$H(s) = \frac{B(s)}{A(s)} = \frac{b(1)s^n + b(2)s^{n-1} + \dots + b(n+1)}{s^n + a(2)s^{n-1} + \dots + a(n+1)}$$

3- Conception des filtres à RIF:

Les filtres à Réponse Impulsionnelle Finie (RIF) présentent les avantages d'être le plus souvent à phase linéaire, stables et d'implémentation facile par rapport aux filtres à RII. Leurs inconvénients majeurs sont un ordre plus élevé et un retard plus grand.

L'idée de base pour l'obtention d'un filtre à RIF est la troncature d'une RII. Pour ceci, on utilise des fenêtres d'observation de durée finie. Différentes fenêtres peuvent être utilisées: Fenêtre rectangulaire, de Hamming, de Hanning, de Blackman...etc. Chacune présente des caractéristiques différentes.

4- Manipulation

4-1 Filtres RII classiques: en utilisant les différents filtres pré-programmés dans Matlab on souhaite concevoir des filtres classiques à RII.

On fixe la fréquence d'échantillonnage à 1000Hz. La fréquence de coupure désirée est à 300Hz. On utilisera les commandes *freqs(b,a)* et *freqz(b,a)* qui traceront automatiquement la réponse fréquentielle du filtre B/A.

a- Visualiser un filtre de *butterworth* d'ordre 9 continu puis discret. Observer et comparer.

b- Visualiser un filtre *elliptique*, d'ordre 9, continu puis discret ayant les caractéristiques suivantes : $R_p = 3$ dB, $R_s = 50$ dB. Observer et comparer.

4-2 Filtres RIF :

Méthode des fenêtres : pour synthétiser un filtre passe-bande, par la méthode des fenêtres, répondant aux spécifications suivantes: $f_{c1} = 500$ Hz, $f_{c2} = 1$ kHz, $f_e = 4$ kHz, $NbCoeff = 47$, Exécuter le programme ci-après puis commenter ?

```
clear all; close all; clc;
fc1 = 500 ; %Début de la BP
fc2 = 1000; %Fin de la BP
fe = 4000; %Fréquence d'échantillonnage
fc1N = fc1/(fe/2); fc2N = fc2/(fe/2); %Normalisation de fc1 et fc2
NbCoeff = 47; %Nombre de coefficients
N=NbCoeff-1 ; %Ordre du filtre
h = FIR1(N,[fc1N fc2N], 'band',rectwin(NbCoeff), 'scale' );
[H F]= freqz(h, 1, 512, fe); % Calcul de H(f)
subplot(2,2,1);hold on; stem(h, 'b' ); grid; %Tracé de la réponse impulsionnelle,
subplot(2,2,2);hold on;plot(F,abs(H)); grid; %Tracé du spectre d'amplitude
subplot(2,2,3);hold on; plot(F,20*log10(abs(H))); grid %Tracé du spectre d'amplitude en db
subplot(2,2,4);hold on; zplane(h,1);grid; %Tracé des pôles et zéros
```


TP III : SIGNAUX ALEATOIRES

ANALYSE ET ESTIMATION SPECTRALES

1- But de la manipulation

On se propose d'étudier les caractéristiques statistiques : moyenne, variance, auto-corrélation, inter-corrélation entre deux signaux numériques $x(n)$ et $y(n)$ et la densité spectrale de puissance (D.S.P.). Nous traiterons le cas où $y(n)$ est le résultat d'un filtrage numérique par différentes fonctions de transfert $H(z)$ attaqué par un bruit blanc $x(n)$ gaussien centré et de variance σ_x^2 .

2- Quelques Rappels théoriques

2.1 Définitions:

La moyenne m d'un signal discret et sa variance σ_x^2 sont données par les espérances mathématiques :

$$m_x = E[x(n)] = \sum_{n=-\infty}^{n=+\infty} p(x(n))x(n) ; \quad \sigma_x^2 = E[(x(n) - m)^2]$$

La fonction d'autocorrélation est donnée par :

$$R_{xx}(k) = E[x(n)x(n+k)] = \lim_{N \rightarrow \infty} \frac{1}{2N+1} \sum_{n=-N}^{n=N} x(n)x(n+k) .$$

La fonction d'intercorrélation est donnée par :

$$R_{xy}(k) = E[x(n)y(n+k)] = \lim_{N \rightarrow \infty} \frac{1}{2N+1} \sum_{n=-N}^{n=N} x(n)y(n+k) .$$

La densité spectrale d'un signal est le carré du module de la transformée de Fourier du signal $S_{xx}(\nu) = |X(\nu)|^2$ ce qui revient à considérer la transformée de Fourier de sa fonction d'auto-corrélation : $S_{xx}(\nu) = TF[R_{xx}(k)]$.

Ces fonctions sont impossibles à calculer sur un ordinateur car elles nécessiteraient un nombre de points infini et donc un temps de calcul infini. En pratique, on calcule les **estimations** de ces grandeurs en supposant que les signaux sont stationnaires. La qualité d'un estimateur est quantifié (ou mesurée) par son **biais** et sa **variance**.

Un estimateur $\hat{g}$ est dit biaisé si son espérance mathématique n'est pas égale à la grandeur g qu'il estime. Le biais d'un estimateur est alors donné par: $B = E(\hat{g}) - g$

La variance d'un estimateur caractérise la dispersion des valeurs autour de $E(\hat{g})$:

$$V = E[(\hat{g} - E(\hat{g}))^2]$$

Un estimateur est dit **consistant** s'il vérifie les deux propriétés suivantes:

- Le biais tend asymptotiquement vers zéro quand $N \rightarrow \infty$.
- La variance tend asymptotiquement vers zéro quand $N \rightarrow \infty$.

2.2 Estimation des propriétés statistiques

Estimation de la moyenne et de la variance.

On suppose que l'on dispose d'une séquence finie de N points du signal $[x_0, x_1, x_2, \dots, x_{N-1}]$

- Estimateur de la moyenne: $\hat{m} = \frac{1}{N} \sum_{n=0}^{N-1} x(n)$.
- Estimateur de la variance : $\hat{\sigma}^2 = \frac{1}{N} \sum_{n=0}^{N-1} (x(n) - m)^2$ (estimateur non biaisé). Si la moyenne m n'est pas connue, on utilise son estimé $\hat{m}$ mais l'estimateur est alors biaisé.

Estimation de la fonction d'auto-corrélation.

On suppose que l'on dispose d'une séquence de N points du signal $[x_0, x_1, x_2, \dots, x_{N-1}]$

Les deux estimateurs classiques de fonction d'auto-corrélation sont donnés par :

- Estimateur biaisé: $\hat{R}_{xx}(k) = \frac{1}{N} \sum_{n=0}^{N-1-|k|} x(n)x(n+k)$
- Estimateur non biaisé : $\hat{R}_{xx}(k) = \frac{1}{N-|k|} \sum_{n=0}^{N-1-|k|} x(n)x(n+k)$

Estimation de la densité spectrale de puissance

On suppose que l'on dispose d'une séquence de N points du signal $[y_0, y_1, y_2, \dots, y_{N-1}]$, il existe différentes méthodes pour estimer la densité spectrale de puissance :

a. Périodogramme:

La méthode la plus simple est la méthode dite de périodogramme. Cette méthode estime la densité spectrale de puissance comme étant le carré du module de la transformée de Fourier du signal $y_N(n)$. On obtient donc:

$$\hat{S}_{yy} = |Y_N(\nu)|^2$$

avec $y_N(n)$ est le signal $y(n)$ pondéré par une fenêtre rectangulaire de largeur N.

b. Méthode de Welch : périodogramme moyenné

On subdivise la séquence en M sous-séquences de longueur L tel que $N = M.L$. On calcule alors le périodogramme de chacune des sous-séquences puis on prend la moyenne.

Pour augmenter le nombre des sous-séquences, il arrive qu'on les fasse chevaucher sur une longueur $D < L$. On aura alors $(M-1)(L-D)+L = N$.

Méthodes du modèle: On cherche à modéliser le signal $y(n)$ en le considérant comme la réponse d'un filtre numérique de fonction de transfert $H(z) = \frac{B(z^{-1})}{A(z^{-1})}$ attaqué par un bruit blanc $x(n)$, gaussien, centré et de variance σ_x^2 . Le modèle peut être de type AR, MA ou ARMA. On calcule alors l'estimé de la densité spectrale par la relation :

$$\hat{S}_{yy}(\nu) = |H(\nu)|^2 S_{xx} \quad \text{où } S_{xx} = \sigma_x^2 \text{ (bruit blanc).}$$

Dans le cas d'un modèle AR d'ordre r on peut construire $H(z) = \frac{1}{A(z^{-1})} = \frac{1}{1 + \sum_{k=1}^r a_k z^{-k}}$ à

partir d'une séquence $y_N(n)$ du signal $y(n)$ en utilisant soit la méthode Yule-Walker soit la méthode de Burg.

La méthode Yule-Walker est basée sur la détermination des coefficients a_k par autocorrelation. La méthode Burg est un algorithme itératif de construction du filtre $H(z)$ d'ordre r à partir de celui de l'ordre $r-1$.

3- Etude théorique

3-1 Corrélation des signaux MA

Un signal $y(n)$ à Moyenne Mobile (Moving Average) est obtenu par passage d'un bruit blanc $x(n)$ dans un filtre à réponse impulsionnelle finie (FIR). On considère le filtre de fonction de transfert $H_1(z) = 0.5 + 0.5z^{-1}$. Montrer que :

- pour les moyennes on a $m_y = H_1(1)m_x$
- $R_{xy}(k) = \frac{1}{2}\sigma_x^2$ (pour $k = 0$) + $\frac{1}{2}\sigma_x^2$ (pour $k = 1$) et zéro ailleurs
- $R_{yy}(k) = \frac{1}{2}\sigma_x^2$ (pour $k = 0$) + $\frac{1}{4}\sigma_x^2$ (pour $k = 1$) + $\frac{1}{4}\sigma_x^2$ (pour $k = -1$) et zéro ailleurs.

3-2 Corrélation des signaux AR

Les signaux AR sont obtenus par passage d'un bruit blanc dans un filtre purement récursif.

On considère le filtre de fonction de transfert $H_2(z) = \frac{1}{1 - a.z^{-1}}$, avec $a = 0.75$ Montrer que :

$$R_{yy}(k) = a^{|k|}.$$

4- Simulation

1. Etude du bruit blanc

- Générer un bruit blanc $x_N(n)$ à distribution gaussienne, de moyenne nulle et de variance $\sigma^2=10$ ayant $N = 2048$ échantillons. (On utilisera la fonction **normrnd**).
- Calculer et tracer l'estimateur $\hat{R}_{xx}(k)$ non biaisé et biaisé de sa fonction d'autocorrélation. (On utilisera les fonctions **xcorr** et **stem**(-N+1:N-1,Rxx)).
- Tracer l'estimation de sa densité spectrale. (On utilisera la fonction **FFT**).
- On considère la séquence $x_L(n)$ formée par les L premiers termes de $x_N(n)$. Calculer les estimateurs de la moyenne et de la variance de la séquence $x_L(n)$ pour les valeurs de L allant de 1 à N. Tracer ces estimateurs. Conclusion.
- Ces résultats confirment-ils que le signal $x_N(n)$ est un bruit blanc.

2. Corrélation des signaux MA

- Filtrer le signal $x_N(n)$ par $H_1(z) = 0.5 + 0.5z^{-1}$. (On utilisera la fonction **filter**).
- Calculer puis tracer les estimateurs non biaisés $\hat{m}_y$, $\hat{R}_{xy}(k)$, $\hat{R}_{yy}(k)$. Comparer avec les valeurs théoriques.

3. Corrélation des signaux AR.

- Filtrer le signal $x_N(n)$ par $H_2(z)$. On obtient un signal $y_N(n)$, Tracer la fonction d'autocorrélation théorique de $y_N(n)$.

On considère la séquence $y_L(n)$ formée par les L premiers termes de $y_N(n)$, afin de montrer l'effet de la longueur de la séquence sur les estimateurs on prendra pour L respectivement les valeurs suivantes $L = 25, 100, 200, 2000$.

- Calculer puis tracer pour chaque valeur de L les estimateurs (biaisé et non biaisé) de la fonction d'autocorrélation. Conclure.

4. Estimation de la densité spectrale de puissance

Filtrer le signal $x_N(n)$ de moyenne nulle par

$$H_3(z) = \frac{1}{1 - 2.2137z^{-1} + 2.9403z^{-2} - 2.1697z^{-3} + 0.9606z^{-4}}. \text{ On obtient un signal } y_N(n).$$

- Calculer la densité spectrale de puissance DSP théorique de $y(n)$. (On utilisera la fonction **freqz**)
- Estimer la DSP par la méthode du periodogramme.
- Estimer la DSP par la méthode de Welch. (l'instruction matlab **pwelch**).

- Estimer la DSP par la méthode Yule-Walker *.(l'instruction matlab **pyulear**).*
- Estimer la DSP par la méthode Burg *.(l'instruction matlab **pburg**).*

Tracer à chaque fois l'estimation et la valeur théorique de la DSP. Comparer et conclure.

5. Exercice : On considère un bruit blanc gaussien de moyenne 0 et de variance 2 (0,2). Sous Matlab, les périodogrammes simple, modifié et moyenné sont appelés par la fonction **psd** (x,NFFT, Fs, window) qui découpe le signal x en tranches de NFFT échantillons les pondère par la fenêtre window et affiche la moyenne de leur TFD.

```
>> x=2*randn(1,512); % 512 échantillons d'un BB gaussien (0,2)
>> Subplot (2,2,1); psd(x(1:128),128,1,ones(1,128)); % périodogramme simple
>> Subplot (2,2,2); psd(x,512,1,ones(1,512)); % périodogramme simple
>> Subplot (2,2,3); psd(x(1:128),128,1,hamming(128)); % périodogramme modifié
>> Subplot (2,2,4); psd(x,128,1,ones(1,128)); % périodogramme moyenné
```

Utiliser ce script et analyser les résultats obtenus, quelle interprétation judicieuse donnerez-vous ?